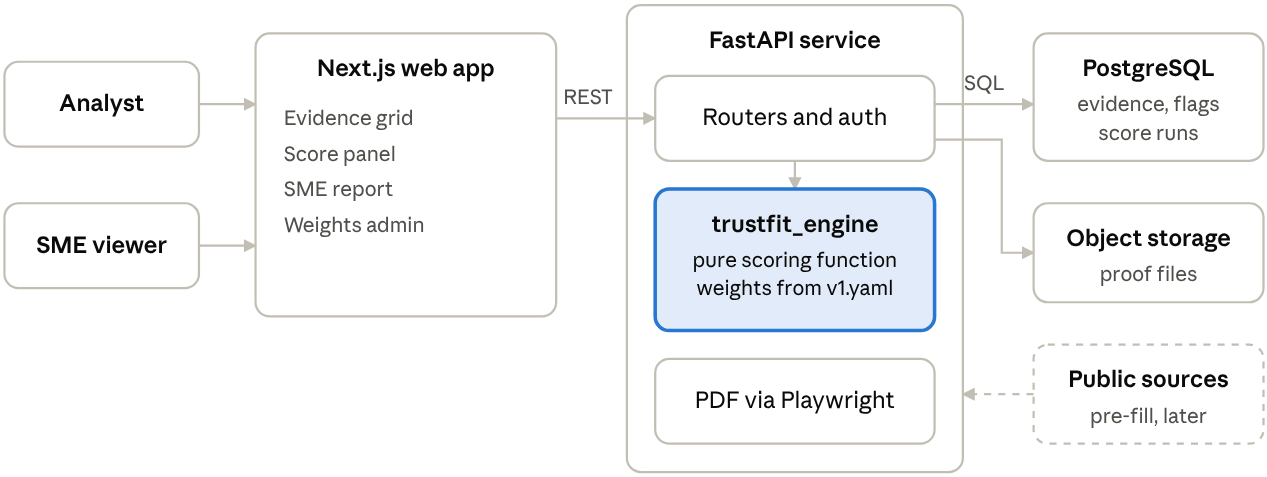
TrustFit TRD (hackathon build)

Oct 24, 2026 · @Harjodh

Architecture overview

Three parts, one rule: the Next.js app only displays results, and every score is computed by the API calling the pure `trustfit_engine` module.

The API runs every score through one pure engine module



TrustFit MVP architecture · web, API with engine, storage

The web app talks to the API over REST; the API stores data in Postgres and proof files in object storage, and prints PDFs from the same report page the SME sees. Public-source fetchers (dashed) come after the MVP.

Tech stack and repo layout

A Python API holds the scoring engine as a pure module, a Next.js app is the front end, and Postgres stores everything; all three run locally with Docker Compose.

Layer	Choice	Why
Front end	Next.js (App Router), TypeScript, Tailwind CSS, shadcn/ui	Fast to build forms and report pages; one-click deploy on Vercel

Layer	Choice	Why
Charts	Recharts	Category bars and dimension gauges with little code
API	FastAPI, Pydantic v2	Typed request and response models; automatic OpenAPI docs for the front end
Engine	Plain Python package `trustfit_engine`	No web or database code inside, so it is easy to unit-test and re-run
Database	PostgreSQL 16, SQLAlchemy 2, Alembic	Relational data with migrations; free hosted tiers such as Neon or Supabase
Auth	JWT with three roles (admin, analyst, viewer)	Enough for a demo; swap for Supabase Auth or Clerk later
PDF	Playwright printing the report page	The PDF always matches the on-screen report
Hosting	Vercel (web), Render or Railway (API), Neon (database)	Free or cheap tiers; a local Docker build is the demo fallback

Repo layout (one monorepo)

```
trustfit/
  apps/
    web/                # Next.js front end
    api/                # FastAPI app: routers, schemas, db models
  packages/
    trustfit_engine/    # pure scoring engine + tests
      config/v1.yaml    # weights, splits, overrides, bands
      tests/fixtures/subros.json
  db/migrations/        # Alembic
  seed/                 # Subros + two more companies
  docker-compose.yml
```

Data model

Ten tables: companies and their evidence, flags and trends, scored against a versioned weight set, with every score run stored.

Table	Key columns	Notes
`companies`	id, name, cin, nse_code, bse_code, sector, created_by	One row per Indian company
`weight_sets`	id, version (e.g. `v1`), status (draft, live), published_at	Only one live version at a time
`parameters`	id (`A1` … `H2`), category, dimension, name, freshness_days (30 or 730), tier	37 rows, seeded from the scoring rules doc
`category_weights`	weight_set_id, role, category, points	3 roles × 8 categories; each role sums to 100
`parameter_splits`	weight_set_id, role (nullable), parameter_id, pct_of_category	Role is null for the default split; set only where a role differs (distributor F3, F4, F5)
`evidence`	id, company_id, parameter_id, level (0/25/50/75/100), verified, source_url, source_date, checked_at, tier, note, analyst_id	One current row per company and parameter; history in `evidence_history`
`flags`	id, company_id, type (knockout, cap, notch, gate, fake_review), code, value, status, evidence_url	value = the cap level or the notch points
`trend_signals`	company_id, metric, direction (improving, stable, worsening), from_value, to_value, period_label	Five rows per company
`score_runs`	id, company_id, role, weight_set_id, inputs_hash, result (JSON), is_what_if, created_by, created_at	Every run kept; what-if runs are never shown to SMEs
`users`	id, email, role (admin, analyst, viewer)	

Rules the database enforces

- `evidence.level` must be one of 0, 25, 50, 75, 100.

- A verified row needs `source_url` and `source_date`.
- Category weights for each role must total 100, and splits for each category must total 100, before a weight set can go live.
- A notch value must lie between −8 and +8.

Scoring engine

The engine is one pure function, `score(inputs, config, today)`: the same inputs and weight version always give the same result, with no database or network calls inside.

Inputs: role; evidence rows (parameter, level, verified, source date, checked date); flags; trend signals; optional what-if overrides. **Config:** the live weight set (`v1.yaml`): category weights per role, parameter splits, freshness days, override codes, bands.

```
KNOCKOUT_CODES = {"struck_off", "sanctions", "insolvency",
"wilful_defaulter", "disqualified_director", "fraud"}

def score(inp, cfg, today):
    rows = []
    for p in cfg.parameters:                                  # 37
parameters
        pts = cfg.weight(inp.role, p.category) * cfg.split(inp.role, p) / 100
        ev = inp.what_if.get(p.id) or inp.evidence.get(p.id)
        usable = ev and ev.verified and is_fresh(ev, p, today)
        if p.id == "D1" and inp.has_flag("fake_review"):
            usable = False                                     # D1 excluded
        rows.append(Row(p, pts, verified=pts if usable else 0,
                        earned=pts * ev.level / 100 if usable else 0))

    verified = sum(r.verified for r in rows)                  # =
confidence, out of 100
    earned = sum(r.earned for r in rows)

    if any(f.code in KNOCKOUT_CODES for f in inp.flags):
        return Result(eligible=False, verdict="Red", reason="knock-out")
    if verified < 40:
        return Result(score=None, verdict="Amber", reason="not enough
verified data")

    base = 100 * earned / verified
    dims = {d: dim_stats(rows, d) for d in ("Integrity", "Financial",
```

```
"Capability")}
    weakest = min(s.score for s in dims.values() if s.verified > 0)
    notches = clamp(sum(f.value for f in inp.flags_of("notch")), -8, 8)
    cap = min([f.value for f in inp.flags_of("cap")], default=100)

    final = round_half_up(min(base + notches, weakest + 15, cap))
    provisional = verified < 60 or any(s.verified_pct < 50 for s in
dims.values())
    return Result(score=final, band=band(final), confidence=verified,
                  dims=dims, categories=cat_stats(rows),
provisional=provisional,
                  verdict=verdict(final, provisional, inp.flags),
                  outlook=outlook(inp.trends), reasons=explain(...))
```

Verdict function, in order: failed gate → Red; final below 55 → Red; provisional, final 55–69, gate pending or fake-review flag → Amber; otherwise Green. Each branch adds a plain-language reason to `reasons`.

Outlook: three or more improving signals → Positive; three or more worsening → Watch; else Stable.

Details that matter

- Freshness: `checked_at` within the parameter's `freshness_days` (30 for registry, court and GST checks; 730 otherwise) and `source_date` within 24 months.
- Rounding: keep full precision until the final score; round half up only for display (82.5 shows as 83).
- Dimension verified % = points verified in the dimension ÷ its total points × 100.
- Every result stores the weight-set version and an `inputs_hash`, so any score can be re-run.

Result JSON (shape)

```
{
  "score": 87, "band": "Very strong", "confidence": 62.25, "provisional":
true,
  "verdict": "Amber", "outlook": "Watch", "eligible": true,
  "dimensions": {"Integrity": {"score": 82.7, "verified_pct": 32.5}, "...":
{}},
  "categories": [{"id": "A", "score_pct": 100, "verified": 3, "points": 15}],
  "overrides": {"notches": [{"code": "strong_shareholder", "value": 3},
{"code": "customer_concentration", "value": -3}],
                "cap": null, "weakest_limit": 97.7},
  "reasons": ["Integrity is 32.5% verified, under the 50% minimum"],
  "weight_set": "v1"
}
```


API design

A small REST API under `/api/v1`, JSON in and out, documented automatically by FastAPI's OpenAPI page at `/docs`.

Method	Path	Role	Purpose
POST	`/auth/login`	any	Email and password → JWT
GET	`/companies`	viewer+	List companies with latest score and verdict
POST	`/companies`	analyst	Create a company
GET	`/companies/{id}`	viewer+	Profile plus latest score per role
GET	`/parameters`	analyst	The 37 parameters with categories and freshness rules (rubric text for analyst and admin only)
GET	`/companies/{id}/evidence`	analyst	All evidence rows, with stale flags
PUT	`/companies/{id}/evidence/{param_id}`	analyst	Create or update one evidence row

Method	Path	Role	Purpose
POST	`/companies/{id}/flags`	analyst	Add a knock-out, cap, notch, gate or fake-review flag
DELETE	`/companies/{id}/flags/{flag_id}`	analyst	Remove a flag
PUT	`/companies/{id}/trends`	analyst	Set the five trend signals
POST	`/companies/{id}/score`	analyst	Body `{role, what_if?}` → result JSON; saved as a run, marked what-if when `what_if` is sent
GET	`/companies/{id}/report?role=`	viewer+	SME-safe report: categories, dimensions, verdict, reasons; no rubric detail
GET	`/companies/{id}/report.pdf?role=`	viewer+	PDF of the report
GET	`/weight-sets`	admin	List versions
POST	`/weight-sets/{id}/publish`	admin	Validate sums and make the version live

Conventions

- Errors return `{"error": {"code", "message", "field"}}`, with 422 for validation and 403 for role.
- `role` is one of `supplier`, `distributor`, `jv_partner`.
- The front end generates its TypeScript types from the OpenAPI schema, so the two never drift.

Frontend pages and components

Seven routes, styled with the pitch deck's colours so the demo and the deck look like one product.

Route	Page	Main components
`/`	Company list	Search box, table with score and verdict chip, "New company" button
`/companies/new`	Company setup	Profile form with CIN check (21 characters), role picker
`/companies/[id]/evidence`	Evidence grid	Accordion per category A–H; per row: level segmented control (0/25/50/75/100), verified switch, source URL, source date, checked date, tier, note; stale badge
`/companies/[id]/flags`	Flags and trends	Override picker by code with evidence link; five trend rows with direction
`/companies/[id]`	Score panel	Role tabs; score card; confidence meter; dimension cards; category bars (pale when under half verified); overrides list; reasons; what-if drawer
`/companies/[id]/report`	SME report	Verdict banner, big score, band, confidence, category bars, outlook, "What turns it Green", PDF button; print stylesheet
`/admin/weights`	Weights admin	Editable weight grid per role, split table, validation errors, publish button

Shared components: `ScoreCard`, `VerdictChip` (Green, Amber, Red), `CategoryBar`, `DimensionCard`, `ConfidenceMeter`, `EvidenceRow`, `FlagPicker`, `WhatIfDrawer`.

Design tokens (from the deck): navy `#15213A`, blue `#1F4E8C`, coral `#E8875A`, paper `#F5F3EE`, muted text `#4A5568`; IBM Plex Sans and IBM Plex Mono.

State: server data via TanStack Query; the score panel re-calls `POST /score` on any evidence change, debounced 300 ms.

Data ingestion and evidence

In the MVP an analyst looks up each fact in a public source and enters it with its link; automation only pre-fills fields for the analyst to confirm.

Category	Typical public sources	MVP method	Later
A Legal & regulatory	MCA21 company master data, GST portal taxpayer search, eCourts, sanctions lists (UN, OFAC, EU)	Manual lookup, 30-day freshness	Sanctions lists downloaded and matched automatically
B Financial health	Annual reports and results filed on NSE/BSE	Manual entry of ratios with filing link	Parse XBRL filings
C Governance	Shareholding patterns, corporate governance reports, MCA director records	Manual	Director checks across MCA records
D Reputation	News, credit rating rationales, verified reviews	Manual; fake-review flag if found	News monitoring
E Payment conduct	Rating rationales (ICRA, CRISIL, CARE), payable days from filings	Manual	Credit bureau data with consent
F Operations	Certification registers (IATF 16949, ISO), plant and capacity disclosures	Manual	Certificate register checks
G Cross-border	Export data in annual reports, foreign shareholders and partners	Manual	Trade data
H ESG	BRSR reports	Manual; stale after 24 months	BRSR parsing

Evidence rules

- A value counts only when verified, with a source link, the document's date and the date checked.
- Analysts can upload a PDF or screenshot as proof; files go to object storage, linked from the evidence row.

- Every change is kept in `evidence_history` with who changed it and when.
- Aggregators such as Screener are fine for leads, but the cited source should be the primary filing.
- No scraping of pages behind a login or against a site's terms.

Security, privacy and compliance

Three roles gate who sees what; the confidential rubric never leaves the analyst side of the API.

Role	Can see	Can change
Viewer (SME)	Published reports: categories, dimensions, verdict, reasons	Nothing
Analyst	Everything for their companies, including rubric text and evidence	Companies, evidence, flags, trends; run scores
Admin	Everything	Weight sets, users

Controls

- Passwords hashed with bcrypt; JWTs expire after 12 hours; HTTPS only in deployment.
- Report endpoints return an SME-safe schema with no rubric cut-offs, fraud rules or analyst notes.
- Secrets (database URL, JWT key) live in environment variables, never in the repo.
- Uploaded proof files are private and served through short-lived signed links.
- Directors' personal data: store only what a check needs, never show it in SME reports, and record why it was collected (India's Digital Personal Data Protection Act, 2023).
- Every report carries the line: "TrustFit Score is not a credit rating or investment advice."

Testing and validation

The Subros fixture is the golden test: if the engine does not reproduce the deck's numbers exactly, nothing ships.

Golden test: Subros as supplier (parameter-level inputs from the Subros example in the scoring rules doc)

Output	Expected
Points verified / earned	62.25 / 54.25
Base score	87.15 → shown as 87
Category scores	A 100, B 87, C 75, D 93, E 75, F 93, G 89, H 83 (%)
Integrity / Financial / Capability	82.7 / 84.8 / 90.5
Verified % by dimension	32.5 / 76.7 / 73.75
Notches, weakest-link limit, cap	0, 97.7, none
Final, band	87, Very strong
Provisional, verdict, outlook	Yes (Integrity under 50%), Amber, Watch
What-if: A1, A3, A4, A5, C1, C2 clean	Verified 82.5, score 90.3 → 90, Green

Rule tests

- Any knock-out code → not eligible, Red, no score.
- Verified below 40 → no score.
- Cap 50 with a base of 80 → final 50.
- Notches adding up to +11 → clamped to +8.
- Weakest dimension 60 with a base of 85 → final 75.
- Evidence checked 31 days ago on a 30-day parameter → dropped and flagged stale.
- Weight-set validation rejects any role whose weights do not total 100.

Property tests (Hypothesis)

- The score always stays within 0–100.
- Raising one parameter's level never lowers the final score.
- Verifying one more parameter never lowers confidence.

Other tests: API tests with pytest and a test database; one Playwright end-to-end run of the demo script, including the PDF export.

Hackathon build plan and deployment

Build the engine and its Subros test first, because everything else displays its output; each phase ends with a check before the next starts. Phase lengths depend on the hackathon window, still to confirm.

Phases

1. **Foundations:** repo, Docker Compose, database schema, `v1.yaml`, 37 parameters seeded. Check: `docker compose up` works on every laptop.
2. **Engine:** scoring function, overrides, verdict, outlook. Check: the Subros golden test passes.
3. **API:** companies, evidence, flags, trends, score and report endpoints. Check: scoring Subros through the API returns the golden result.
4. **Front end:** evidence grid, score panel, SME report. Check: the demo script runs click by click.
5. **P1 polish:** role switcher, what-if, freshness flags, PDF. Check: the full acceptance list in the PRD is ticked.
6. **Demo hardening:** two more companies seeded, deploy, backup video. Check: two clean dry runs, one of them offline.

Team split (four people assumed)

Person	Owns	Hands over
1 Engine lead	`trustfit_engine`, `v1.yaml`, tests	Result schema in phase 2
2 Backend	FastAPI, database, auth, PDF	OpenAPI types in phase 3
3 Frontend	Next.js pages and components	Demo path in phase 4
4 Data and pitch	Subros and two more companies' evidence with sources, demo script, deck	Seed files in phase 2, script in phase 6

Deployment

- Local: `docker compose up` runs web, API and Postgres with seeded data; this is the demo fallback.
- Hosted: web on Vercel, API on Render or Railway, Postgres on Neon; environment variables for secrets.
- Seeding: `python -m seed` loads the weight set, parameters and the three companies.

- CI: GitHub Actions runs engine, API and end-to-end tests on every push.